OBJECT™ TECHNOLOGIES

Interfaces

Need of interface

- java programming language does not support multiple inheritance, using interfaces we can achieve this as a class can implement more than one interfaces, however it cannot extend more than one classes.
- interfaces facilitate the key concepts in OOP like abstraction, inheritance and polymorphism. In addition, interfaces add flexibility and re-usability for software components.
- Java Interface also **represents IS-A relationship**.

What is interface

- *Interface* is a reference type, similar to a class, that can contain *only* constants, method signatures, default methods, static methods, and nested types. Method bodies exist only for default methods and static methods.
- An interface is implicitly abstract. You do not need to use the **abstract** keyword while declaring an interface.
- Each method in an interface is also implicitly abstract, so the abstract keyword is not needed.
- Methods in an interface are implicitly public.
- the variables declared in an interface are public, static & final by default

Defining an Interface

OBJECT TECHNOLOGIES

- An interface declaration consists of modifiers, the keyword interface, the interface name, a comma-separated list of parent interfaces (if any), and the interface body.

```
public interface GroupedInterface extends Interface1, Interface2, Interface3 {

    // constant declarations

    // base of natural logarithms
    double E = 2.718282;

    // method signatures
    void doSomething (int i, double x);
    int doSomethingElse(String s);
}
```

- The public access specifier indicates that the interface can be used by any class in any package. If you do not specify that the interface is public, then your interface is accessible only to classes defined in the same package as the interface

The Interface Body

OBJECT

- The interface body can contain abstract methods, default methods, and static methods. An abstract method within an interface is followed by a semicolon, but no braces (an abstract method does not contain an implementation).
- Default methods are defined with the default modifier, and static methods with the static keyword.
- All abstract, default, and static methods in an interface are implicitly public, so you can omit the public modifier.
- In addition, an interface can contain constant declarations. All constant values defined in an interface are implicitly public, static, and final. These modifiers can be omitted.

Interface Implementation

OBJECT TECHNOLOGIES

- To use an interface, you write a class that *implements* the interface. When an instantiable class implements an interface, it provides a method body for each of the methods declared in the interface
- Class implementing any interface must implement all the methods, otherwise the class should be declared as "abstract".
- One class can implement more than one interface so implements clause can be followed by comma separated list of interfaces
- By convention, the implements clause follows the extends clause, if there is one.

Interface Implementation

OBJECT TECHNOLOGIES

```
interface MyInterface
{
    public void method1();
    public void method2();
}
class XYZ implements MyInterface
{
  public void method1()
  {
      System.out.println("implementation of method1");
  }
  public void method2()
  {
      System.out.println("implementation of method2");
  }
  public static void main(String arg[])
  {
      MyInterface obj = new XYZ();
      obj. method1();
  }
```

Interface Inheritance

OBJECT TECHNOLOGIES

- An interface can extend other interfaces, just as a class subclass or extend another class. However, whereas a class can extend only one other class, an interface can extend any number of interfaces.
- The interface declaration includes a comma-separated list of all the interfaces that it extends.
- A class should provide implementations of all the methods in the interface even derived from its super-interface

```
public interface Inf1{
    public void method1();
}
public interface Inf2 extends Inf1 {
    public void method2();
}
public class Demo implements Inf2{
  public void method1(){
    //Implementation of method1
  }
  public void method2(){
    //Implementation of method2
  }
}
```

Key Points

OBJECT TECHNOLOGIES

- We can't instantiate an interface in java.
- Interfaces will not have any constructors.
- Class implementing any interface must implement all the methods, otherwise the class should be declared as "abstract".
- Interface cannot be declared as private, protected or transient.
- All the interface methods are by default **abstract and public**.
- Variables declared in interface are **public, static and final** by default.
- Interface variables must be initialized at the time of declaration otherwise compiler will through an error.
- Inside any implementation class, you cannot change the variables declared in interface because by default, they are public, static and final
- Variable names conflicts can be resolved by interface name

Key points

- If there are **two or more same methods** in two interfaces and a class implements both interfaces, implementation of the method once is enough.
- A class cannot implement two interfaces that have methods with same name but different return type

Java 8 Interface Improvement

OBJECT TECHNOLOGIES

- Designing interfaces have always been a tough job because if we want to add additional methods in the interfaces, it will require change in all the implementing classes. As interface grows old, the number of classes implementing it might grow to an extent that it's not possible to extend interfaces.
- If a new functionality is added in the existing interface, all the classes implementing interface will break.
- So a facility of static and default methods is given to make improvement in the existing interface

Default Methods

OBJECT TECHNOLOGIES

- Default methods enable you to add new functionality to the interfaces of your libraries and ensure binary compatibility with code written for older versions of those interfaces.
- a method definition in an interface is a default method with the default keyword at the beginning of the method signature. All method declarations in an interface, including default methods, are implicitly public, so you can omit the public modifier.

```
public interface Interface2 {

        void method2();

        default void log(String str){
                System.out.println("I2 logging::"+str);
        }

}
```

Default Methods

OBJECT TECHNOLOGIES

- When you extend an interface that contains a default method, you can do the following:
  - Not mention the default method at all, which lets your extended interface inherit the default method as it is and use it
  - Redeclare the default method, which makes it abstract.
  - Redefine the default method, which overrides it.

Static methods

OBJECT TECHNOLOGIES

- In addition to default methods, you can define static methods in interfaces.
- This makes it easier for you to organize helper methods in your libraries; you can keep static methods specific to an interface in the same interface rather than in a separate class.
- We can't override static methods in the implementation classes.
- Java interface static method is visible to interface methods only
- We can use interface static methods using interface name.
- We can't define interface static method for Object class methods, we will get compiler error

Marker interfaces

OBJECT TECHNOLOGIES

- An interface with no methods in it is referred to as a tagging or marker interface.
- For example Serializable, Clonable, EventListener, Remote(java.rmi.Remote) are tag interfaces.
- Two basic design purposes :
  - Creates a common parent – As with the EventListener interface, which is extended by dozens of other interfaces in the Java API, we can use a tagging interface to create a common parent among a group of interfaces. For example, when an interface extends EventListener, the JVM knows that this particular interface is going to be used in an event delegation scenario.
  - Adds a data type to a class – This situation is where the term, tagging comes from. A class that implements a tagging interface does not need to define any methods (since the interface does not have any), but the class becomes an interface type through polymorphism.

Nested Interfaces

OBJECT TECHNOLOGIES

- An interface which is declared inside another interface or class is called nested interface. They are also known as inner interface. For example Entry interface in collections framework is declared inside Map interface, that's why we don' use it directly, rather we use it like this: Map.Entry.

Comparision between Abstract Classes and Interfaces

Abstract classes are similar to interfaces. You cannot instantiate them, and they may contain a mix of methods declared with or without an implementation. However, with abstract classes, you can declare fields that are not static and final, and define public, protected, and private concrete methods. With interfaces, all fields are automatically public, static, and final, and all methods that you declare or define (as default methods) are public. In addition, you can extend only one class, whether or not it is abstract, whereas you can implement any number of interfaces.

Which should you use, abstract classes or interfaces?

Consider using abstract classes if any of these statements apply to your situation:

- You want to share code among several closely related classes.
- You expect that classes that extend your abstract class have many common methods or fields, or require access modifiers other than public (such as protected and private).
- You want to declare non-static or non-final fields. This enables you to define methods that can access and modify the state of the object to which they belong.

Consider using interfaces if any of these statements apply to your situation:

- You expect that unrelated classes would implement your interface. For example, the interfaces Comparable and Cloneable are implemented by many unrelated classes.
- You want to specify the behavior of a particular data type, but not concerned about who implements its behavior.
- You want to take advantage of multiple inheritance of type.

Assignments

1. Create interface Drawable having members as PI, methods like drawShape, calculateArea. Implement this interface to create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code
2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)
   Create the class MyNumber implemeting this interface. Accept the number from the user to create instance of MyNumber

Assignments

1. Create interface Drawable having members as PI, methods like drawShape, calculateArea. Implement this interface to create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code
2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)
   Create the class MyNumber implemeting this interface. Accept the number from the user to create instance of MyNumber and call above methods.
3. Create interface ITraveller. It has method public double calculateTA() and has a static final member DA (in percent of basic salary) Modify Emp hierarchy for implementing this interface only for those employees who are allowed to travel for their job work(only for SalesManager and Programmer). From the application display information about empid, no of days travelled and travelling allownace given to all employees.
4. Create an interface Comparer having method like public int compare(Object o1, Object o2). Create different implementing classes as RollComparar, NameComparer and MeritComparer. Implement compare method accordingly. Create a class Student having data members as rollno, name and marks. Create array of students and sort on the basis of either rollno, name or marks based on user input.(accept the choice of sorting from user)
5. Create interface StringOperations having methods
   void reverse(String str)
   void toUpperCase(String str)
   int length(String str)
   boolean isPallindrome(String str)
   String append(String str1,String str2)
   Implement above interface in the class and override all the methods. Accept the string from user in main method and perform all operations on that string
   (Hint : Do not use built in methods of String class, use charAt(int index) method in class String which returns individual character from the given index)

extra :

1. Create array of 5 intergers. Iterate thr the array and swap the values if needed to sort the array. Display the sorted array

extra :

1. Create array of 5 intergers. Iterate thr the array and swap the values if needed to sort the array. Display the sorted array.

Assignments-1

1. Create interface Drawable having members as PI, methods like drawShape, calculateArea. Implement this interface to create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code
2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)
   Create the class MyNumber implemeting this interface. Accept the number from the user to create instance of MyNumber and call above methods.
3. Create interface ITraveller. It has method public double calculateTA() and has a static final member DA (in percent of basic salary) Modify Emp hierarchy for implementing this interface only for those employees who are allowed to travel for their job work(only for SalesManager and Programmer). From the application display information about empid, no of days travelled and travelling allownace given to all employees.
4. Create an interface Comparer having method like public int compare(Object o1, Object o2). Create different implementing classes as RollComparar, NameComparer and MeritComparer. Implement compare method accordingly. Create a class Student having data members as rollno, name and marks. Create array of students and sort on the basis of either rollno, name or marks based on user input.(accept the choice of sorting from user)

Assignments-2

1. Create interface Drawable having members as PI, methods like drawShape, calculateArea. Implement this interface to create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code
2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)

create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code

2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)
   Create the class MyNumber implemeting this interface. Accept the number from the user to create instance of MyNumber and call above methods.
3. Create interface ITraveller. It has method public double calculateTA() and has a static final member DA (in percent of basic salary) Modify Emp hierarchy for implementing this interface only for those employees who are allowed to travel for their job work(only for SalesManager and Programmer). From the application display information about empid, no of days travelled and travelling allownace given to all employees.
4. Create an interface Comparer having method like public int compare(Object o1, Object o2). Create different implementing classes as RollComparar, NameComparer and MeritComparer. Implement compare method accordingly. Create a class Student having data members as rollno, name and marks. Create array of students and sort on the basis of either rollno, name or marks based on user input.(accept the choice of sorting from user)

Assignments-2

1. Create interface Drawable having members as PI, methods like drawShape, calculateArea. Implement this interface to create concrete classes for different shapes like circle, rectangle, triangle etc.Override required functionality and verify from application code
2. Create interface IntOperations having methods as
   public boolean isOdd(int num)
   public boolean isEven(int num)
   public boolean isPrime(int num)
   public double calFactorial(int num)
   Create the class MyNumber implemeting this interface. Accept the number from the user to create instance of MyNumber and call above methods.